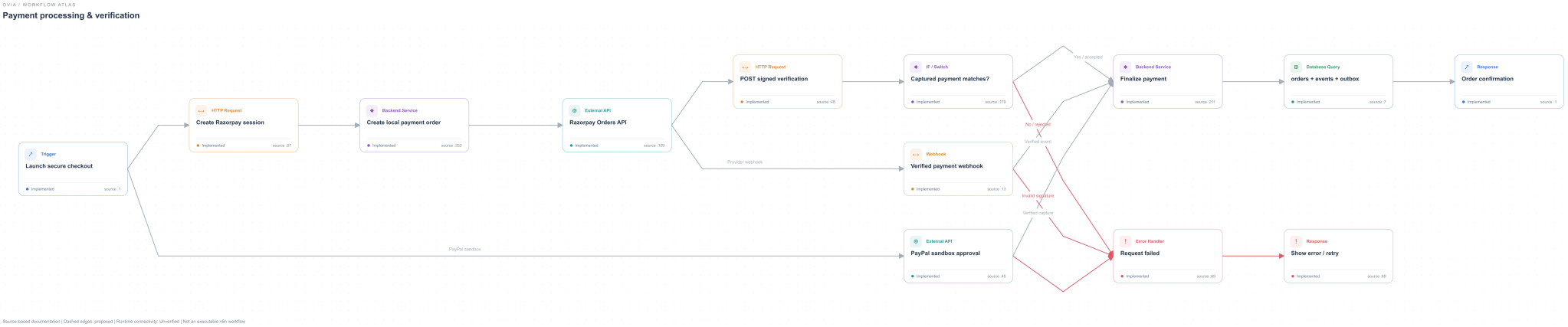
Payment processing & verification

Architecture overview. Readable node and connection sheets follow.



Payment processing & verification

Readable detail sheet 1/4. Node numbers and page references preserve every directed connection.

#01 / Trigger / Implemented

Launch secure checkout

Customer confirms the reviewed quote and starts provider checkout.

src/app/checkout/page.tsx:1

Next -> #2 (p2); PayPal sandbox -> #13 (p5)

#02 / HTTP Request / Implemented

Create Razorpay session

Validate input, owner, origin and rate limits before beginRazorpayCheckout.

src/app/api/v1/payments/razorpay/[[...action]]/route.ts:27

Next -> #3 (p2)

#03 / Backend Service / Implemented

Create local payment order

createOrder reserves stock and creates a durable order; provider order data is attached.

src/server/razorpay-checkout.ts:322

Next -> #4 (p2)

#04 / External API / Implemented

Razorpay Orders API

Authenticated provider requests create and retrieve payment orders. Runtime configuration and connectivity are Unverified.

src/server/razorpay-checkout.ts:109

Next -> #5 (p3); Provider webhook -> #12 (p4)

Payment processing & verification

Readable detail sheet 2/4. Node numbers and page references preserve every directed connection.

#05 / HTTP Request / Implemented

POST signed verification

verifyRazorpayPayment validates returned payment identifiers and signature.

src/app/api/v1/payments/razorpay/[[...action]]/route.ts:48

Next -> #6 (p3)

#06 / IF / Switch / Implemented

Captured payment matches?

Amount, currency, remote order identity and capture status must match the local order.

src/server/razorpay-checkout.ts:179

Yes / accepted -> #7 (p3); No / rejected -> #10 (p4)

#07 / Backend Service / Implemented

Finalize payment

Duplicate events are checked; timely captures become paid, late captures require refund review.

src/server/commerce.ts:211

Next -> #8 (p3)

#08 / Database Query / Implemented

orders + events + outbox

Payment metadata remains on orders; event IDs protect replay; confirmation mail is queued only for confirmed purchases.

src/server/commerce.ts:7

Next -> #9 (p4)

Payment processing & verification

Readable detail sheet 3/4. Node numbers and page references preserve every directed connection.

#09 / Response / Implemented

Order confirmation

Customer receives verified order state, never an assumed successful capture.

src/app/orders/[id]/confirmation/page.tsx:1

END: no outgoing connection

#10 / Error Handler / Implemented

Request failed

Validation, access, conflict or service errors are returned to the caller. Status and error shape depend on the specific route.

src/server/http.ts:69

Next -> #11 (p4)

#11 / Response / Implemented

Show error / retry

Failure end state. Caller receives the actual error; a retry may be offered by the UI.

src/server/http.ts:69

END: no outgoing connection

#12 / Webhook / Implemented

Verified payment webhook

Stripe, Razorpay and PayPal dispatch; signature verification precedes reconciliation.

src/app/api/webhooks/[provider]/route.ts:13

Verified event -> #7 (p3); Invalid signature -> #10 (p4)

Payment processing & verification

Readable detail sheet 4/4. Node numbers and page references preserve every directed connection.

#13 / External API / Implemented

PayPal sandbox approval

Create sandbox order, redirect approval, capture and verify provider response. Runtime configuration and connectivity are Unverified.

src/server/paypal-checkout.ts:45

Verified capture -> #7 (p3); Next -> #10 (p4)

Payment processing & verification

Complete directed connection register / no omitted success, failure or proposed edges

#01 Launch secure checkout -> #02 Create Razorpay session | Next

#02 Create Razorpay session -> #03 Create local payment order | Next

#03 Create local payment order -> #04 Razorpay Orders API | Next

#04 Razorpay Orders API -> #05 POST signed verification | Next

#05 POST signed verification -> #06 Captured payment matches? | Next

#06 Captured payment matches? -> #07 Finalize payment | Yes / accepted

#07 Finalize payment -> #08 orders + events + outbox | Next

#08 orders + events + outbox -> #09 Order confirmation | Next

#06 Captured payment matches? -> #10 Request failed | No / rejected | FAILURE

#10 Request failed -> #11 Show error / retry | Next | FAILURE

#04 Razorpay Orders API -> #12 Verified payment webhook | Provider webhook

#12 Verified payment webhook -> #07 Finalize payment | Verified event

#12 Verified payment webhook -> #10 Request failed | Invalid signature | FAILURE

#01 Launch secure checkout -> #13 PayPal sandbox approval | PayPal sandbox

#13 PayPal sandbox approval -> #07 Finalize payment | Verified capture

#13 PayPal sandbox approval -> #10 Request failed | Next | FAILURE

Note: Source implementation is documented; deployed configuration and provider connectivity are not verified.

Note: PayPal uses a separate implemented sandbox create/approve/capture flow. Legacy Stripe checkout is guarded to test credentials.